



Уголок сельского джависта

Я — сельский джавист, и друзья меня зовут Шурик. А это мой профессиональный блог о Java, Kotlin, Spring и многом другом.

Redis: основные типы данных и операции

Redis — это популярная нереляционная база данных, реализующая хранилище типа «Ключ — Значение». Данная статья посвящена основным типам данных и операциям и является вступи-



тельной к серии статей о Redis, в рамках которой будет рассматриваться и работа с Redis при помощи Spring

Data.

Что такое Redis

Redis — это хранилище данных типа «Ключ — Значение» (аналог Map в Java, Kotlin и других языках программирования). В отличие от более традиционных реляционных баз данных вроде MySQL и PostgreSQL или нереляционных вроде MongoDB и Cassandra, в Redis

отсутствуют таблицы и коллекции, а вся база данных представляет собой справочник, состоящий из ключей и значений. По умолчанию Redis может хранить до $4\,294\,967\,295$ ($2^{32}-1$) пар «ключ — значение».

При этом Redis является высокопроизводительной базой данных: заявляется о производительности в 100 000 GET/SET-команд в секунду на сервере начального уровня под управлением GNU/Linux, что делает Redis весьма мощным инструментом для ряда задач, вроде кеширования, индексации, организации поиска и аналитики. Обеспечивается такая производительность, в том числе, за счёт хранения данных в оперативной памяти.

Основные типы данных

Redis OSS поддерживает следующие типы данных для значений: строки, хеш-таблицы, списки, множества, потоки, битовые карты и поля, а так же геопространственные структуры.

Кроме этого Redis Stack и Redis Enterprise поддерживают JSON, вероятностные структуры данных и временные ряды. Предлагаю вкратце разобратся с основными типами данных.

Строки (String)

Строки являются основным типом данных в Redis для значений, а для ключей — и вовсе единственным. С точки зрения разработчиков программного обеспечения на языках программиро-

вания вроде Java, Kotlin и других, Redis в целом можно представить себе как `Map<String, String>`.

По умолчанию максимальный размер строкового значения составляет 512Мб.

Основные операции

- `set <key> <value>` — устанавливает значение для указанного ключа

```
1 # Присваивание значения bar ключу foo
2 127.0.0.1:6379> set foo bar
3 OK
```

- `get <key>` — возвращает текущее значение или `nil`

```
1 # Получение несуществующего значения
2 127.0.0.1:6379> get baz
3 (nil)
4
5 # Получение существующего значения
```

```
6 127.0.0.1:6379> get foo
7 "bar"
```

- `del <key>` — удаляет значение, применима ко всем ключам, вне зависимости от типа значения

```
1 # Удаление значения
2 127.0.0.1:6379> del foo
3 (integer) 1
```

Строки в Redis могут себя вести как числа, битовые карты и поля, более подробно это и остальные операции будут рассмотрены в одной из следующих статей.

Хеш-таблицы (Hash table)

Хеш-таблицы представляют собой структуру типа «ключ — значение». Redis позволяет хранить до $4\,294\,967$

295 ($2^{32}-1$) пар «ключ — значение» в хеш-таблице.

Аналог хеш-таблиц в Java — HashMap.

Основные операции

- `hset <key> <hashkey> <hashvalue>` — установка значения в хеш-таблице, при этом пары `hashkey-hashvalue` могут повторяться. Если значение присваивается существующей хеш-таблице, то существующие значения будут заменены новыми.

```
1 # Пример установки ключа manufacturer
2 127.0.0.1:6379> hset car manufacturer
3 (integer) 1
4
5 # Пример установки новых значений для
6 127.0.0.1:6379> hset car manufacturer
7 (integer) 2 # Количество добавленных к
8
9 # Пример замены значений для ключей ma
```

```
10 127.0.0.1:6379> hset car manufacturer
11 (integer) 0
```

- `hget <key> <hashkey>` — получение значения из хеш-таблицы

```
1 127.0.0.1:6379> hget car manufacturer
2 "Ford"
```

- `hmget <key> <hashkey1> <hashkey2>` — получение значений для одного и более ключей из хеш-таблицы

```
1 127.0.0.1:6379> hmget car manufacturer
2 "Ford"
3 "2024"
```

- `hgetall <key>` — получение всех ключей и значений указанной хеш-таблицы

```
1 127.0.0.1:6379> hgetall car
2 1) "manufacturer"
3 2) "Ford"
4 3) "model"
5 4) "Expedition"
6 5) "year"
7 6) "2024"
```


- `hdel <key> <hashkey1> <hashkey2> —`
удаление указанных ключей из хеш-таблицы

```
1 127.0.0.1:6379> hdel car year
2 (integer) 1
```

Списки (List)

Списки представляют собой упорядоченные последовательности элементов и во многих языках программирования реализованы с использованием массивов (например, `ArrayList` в Java), но в Redis списки являются именно связанными списками, благодаря чему при помощи их можно реализовывать, например, логику очередей. Списки в Redis могут содержать дублирующиеся элементы.

Аналог списков Redis в Java — LinkedList.

Основные операции

- `lpush <key> <element1> <element2> <elementn>` (*left push*) — добавление элементов в начало (левый край) списка
- `rpush <key> <element1> <element2> <elementn>` (*right push*) — добавление элементов в конец (правый край) списка

При вставке нескольких элементов в начало списка стоит помнить, что элементы вставляются в список поочерёдно, а не вместе. Если вызвать операцию `lpush list element1 element2`, то фактический порядок элементов в

списке будет `element2` `element1`, так как сначала в начало списка будет добавлен `element1`, а затем в начало же списка будет добавлен `element2`. При вызове операции `rpush` порядок вставляемых элементов не изменяется.

```
1 127.0.0.1:6379> lpush manufacturers Ford
2 (integer) 2 # количество элементов в списке
3
4 127.0.0.1:6379> rpush manufacturers GMC
5 (integer) 3 # количество элементов в списке
```

- `llen <key>` (*list length*) — получение количества элементов списка

```
1 127.0.0.1:6379> llen manufacturers # количество элементов в списке
2 (integer) 3
```

- `lrange <key> <start> <stop>` (*list range*) — получение элементов списка без их удаления. При помощи `start` и `stop` можно выбирать диапазон получаемых значений, первый элемент имеет позицию 0,

диапазон указывается включительно. В качестве верхней границы можно указывать число, превышающее реальное количество элементов в списке или -1, в этом случае будут возвращены все элементы списка.

```
1 127.0.0.1:6379> lrange manufacturers 0
2 1) "Chevrolet"
3 2) "Ford"
4 3) "GMC"
5
6 127.0.0.1:6379> lrange manufacturers 0
7 1) "Chevrolet"
8 2) "Ford"
9
10 127.0.0.1:6379> lrange manufacturers 1
11 1) "Ford"
```

- `lpop <key> <count>` (*left pop*) — извлечение элементов из начала списка. Возвращает и удаляет указанное количество элемент из начала списка (по умолчанию 1).
- `rpop` (*right pop*) — извлечение элемента из конца списка. Возвращает

и удаляет указанное количество элемент из начала списка (по умолчанию 1). При извлечении двух и более элементов из конца списка элементы будут возвращены в обратном порядке, так как извлекаться будут по очереди из конца списка.

```
1 127.0.0.1:6379> lpop manufacturers # ИЗ
2 "Chevrolet"
3
4 127.0.0.1:6379> rpop manufacturers 2 #
5 1) "GMC"
6 2) "Ford"
7
8 127.0.0.1:6379> rpop manufacturers # ИЗ
9 (nil)
```

- `blpop <key> <timeout>` (*blocking left pop*) — блокирующее извлечение элемента из начала списка. Возвращает и удаляет элемент из начала списка. Если список пуст, то дожидается появления первого элемента указанное количество секунд.

```
1  # блокирующее извлечение элемента из
2  127.0.0.1:6379> blpop manufacturers 3
3  (nil) # элемент не появился
4  (3.06s) # затраченное время
5
6  # блокирующее извлечение элемента из
7  127.0.0.1:6379> blpop manufacturers 5
8  1) "manufacturers"
9  2) "Chevrolet" # элемент появился
10 (2.56s) # затраченное время
```

- `ltrim <key> <start> <stop>` (*list trim*)
— усечение списка до указанного диапазона, диапазон задаётся от 0, границы указываются включительно

```
1  # Добавляем в список 5 элемента
2  127.0.0.1:6379> lpush manufacturers Fo
3  (integer) 5
4
5  # Список
6  127.0.0.1:6379> lrange manufacturers 0
7  1) "Dodge"
8  2) "Jeep"
9  3) "GMC"
10 4) "Chevrolet"
11 5) "Ford"
12
13 # Усекаем список до элементов 2-4
14 127.0.0.1:6379> ltrim manufacturers 2
```

```
15 OK
16
17 # Полученный список
18 127.0.0.1:6379> lrange manufacturers 0
19 1) "GMC"
20 2) "Chevrolet"
21 3) "Ford"
```

- `lmove <source> <destination>`
`<left|right> <left|right> (list move)`
— атомарное перемещение элемента из одного списка в другой.

```
1 # Добавляем в список 5 элемента
2 127.0.0.1:6379> lpush manufacturers Fo
3 (integer) 5
4
5 # Список
6 127.0.0.1:6379> lrange manufacturers 0
7 1) "Dodge"
8 2) "Jeep"
9 3) "GMC"
10 4) "Chevrolet"
11 5) "Ford"
12
13 # Переместим первый элемент manufactur
14 127.0.0.1:6379> lmove manufacturers us
15 "Dodge" # перемещённый элемент
16
17 # Состояние списков
18 127.0.0.1:6379> lrange manufacturers 0
```

```
19 1) "Jeep"
20 2) "GMC"
21 3) "Chevrolet"
22 4) "Ford"
23 127.0.0.1:6379> lrange usmanufactur
24 1) "Dodge"
25
26 # Переместим последний элемент manufac
27 127.0.0.1:6379> lmove manufacturers us
28 "Ford" # перемещённый элемент
29
30 # Состояние списков
31 127.0.0.1:6379> lrange manufacturers 0
32 1) "Jeep"
33 2) "GMC"
34 3) "Chevrolet"
35 127.0.0.1:6379> lrange usmanufacturers
36 1) "Ford"
37 2) "Dodge"
```

- `blmove <source> <destination>`
`<left|right> <left|right>`
`<timeout> (blocking list_move)` — атомарное блокирующее перемещение элемента из одного списка в другой. Поведение аналогично `lmove`, но появляется таймаут ожидания, если исходный список пустой.


```
1 # Переместим первый элемент manufacture
2 127.0.0.1:6379> blmove manufacturers us
3 "Dodge" # перемещённый элемент
4
5 # Попробуем переместить элемент из пус
6 127.0.0.1:6379> blmove manufacturers us
7 (nil) # перемещаемый элемент отсутствует
8 (5.06s) # затраченное время
```

Множества (Set)

Множество (set) представляют собой неупорядоченный набор уникальных элементов.

Аналог множества Redis в Java — HashSet.

Основные операции

- `sadd <key> <element1> <element2> <elementN>` (*set add*) — добавление эле-

МЕНТОВ В МНОЖЕСТВО

```
1 # Добавление 5 элементов в множество ma
2 127.0.0.1:6379> sadd manufacturers Ford
3 (integer) 5 # количество добавленных эл
```

- `scard <key>` (*set cardinality*) — получение количества элементов множества

```
1 127.0.0.1:6379> scard manufacturers
2 (integer) 6
```

- `smembers <key>` (*set members*) — вывод элементов множества

```
1 127.0.0.1:6379> smembers manufacturers
2 1) "Ford"
3 2) "Chevrolet"
4 3) "Dodge"
5 4) "Jeep"
6 5) "GMC"
```

- `srem <key> <element1> <element2> <elementN>` (*set remove*) — удаление элементов из множества

```
1 # Удаление элементов Jeep и Ford из мно
2 127.0.0.1:6379> srem manufacturers Jeep
3 (integer) 2 # количество удалённых элем
4
5 # Итоговое множество
6 127.0.0.1:6379> smembers manufacturers
7 1) "Chevrolet"
8 2) "Dodge"
9 3) "GMC"
```

- `sismember <key> <value>` (*set is member*) — проверка, является ли значение элементом указанного множества

```
1 # Проверка, является ли Chevrolet элеме
2 127.0.0.1:6379> sismember manufacturers
3 (integer) 1 # да
4
5 # Проверка, является ли Ford элементом
6 127.0.0.1:6379> sismember manufacturers
7 (integer) 0 # нет
```

- `sinter <key1> <key2> <keyN>` (*set intersection*) — получение элементов, которые встречаются во всех указанных множествах

```
1 # Добавление 6 элементов в множество m
2 127.0.0.1:6379> sadd manufacturers For
3 (integer) 6
4
5 # Добавление 3 элементов в множество u
6 127.0.0.1:6379> sadd usmanufacturers F
7 (integer) 3
8
9 # Поиск общих элементов
10 127.0.0.1:6379> sinter manufacturers u
11 1) "Ford"
12 2) "Chevrolet"
13 3) "GMC"
```

Сортированные множества (Sorted Set)

Сортированные множества представляют собой наборы уникальных элементов, отсортированных по весу (score). Если несколько элементов имеют одинаковый вес, то они сортируются по алфавиту.

В некоторой мере аналогом сортированных множеств в Java является SortedSet и его реализации.

Основные операции

- `zadd <key> <nx|xx> <gt|lt> <ch> <incr> <score> <member>` — добавление элемента в сортированное множество. Аргументы операции:
 - `nx` — только добавлять новые элементы, не изменяя существующие
 - `xx` — только изменять существующие элементы, не добавляя новые
 - `gt` — обновлять элементы только, если новый вес больше существующего
 - `lt` — обновлять элементы только, если новый вес меньше

существующего

- `ch` — показывать количество изменённых элементов, а не добавленных
- `incr` — увеличивать существующий вес на указанный (может быть использован только с одной парой вес-элемент)
- `score` — вес элемента
- `member` — элемент

Команда `zadd` может принимать несколько пар вес-элемент (`score member`), если не используется аргумент `incr`. Аргументы `nx`, `gt` и `lt` являются взаимоисключающими.

```
1 # Добавление в множество трёх элементов
2 127.0.0.1:6379> zadd manufacturers 10
3 (integer) 3
4
5 # Содержимое множества
6 127.0.0.1:6379> zrange manufacturers 0
7 1) "Chevrolet"
```

```
8 2) "10"
9 3) "GMC"
10 4) "20"
11 5) "Ford"
12 6) "30"
13
14 # Изменение весов элементов
15 127.0.0.1:6379> zadd manufacturers 30
16 (integer) 0 # количество добавленных э
17
18 # Содержимое множества
19 127.0.0.1:6379> zrange manufacturers 0
20 1) "GMC"
21 2) "10"
22 3) "Ford"
23 4) "20"
24 5) "Chevrolet"
25 6) "30"
26
27 # Увеличение веса элемента Ford на 20
28 127.0.0.1:6379> zadd manufacturers inc
29 "40" # Итоговый вес
30
31 # Содержимое множества
32 127.0.0.1:6379> zrange manufacturers 0
33 1) "GMC"
34 2) "10"
35 3) "Chevrolet"
36 4) "30"
37 5) "Ford"
38 6) "40"
```

- `zcard <key>` — получение количества элементов множества

```
1 127.0.0.1:6379> zcard manufacturers
2 (integer) 3 # количество элементов
```

- `zcount <key> <score-min> <score-max>` — получение количества элементов множества в указанном диапазоне весов. Границы диапазона указываются включительно.

```
1 127.0.0.1:6379> zcount manufacturers 10
2 (integer) 2 # количество элементов
```

- `zrange <key> <start> <stop> <byscore | bylex> <rev> <limit offset count> <withscores>` — получение элементов множества. Аргументы операции:
- `start` и `stop` — диапазон элементов множества, числа если используется сортировка по весу, иначе элементы списка с ведущей [или (для указа-

ния включения или исключения
элемента в диапазоне, либо - + —
для использования всего диапазона.

- `byscore` | `bylex` — сортировка по весу или алфавитному порядку
- `rev` — получение элементов в обратном порядке
- `limit offset count` — дополнительное ограничение получаемого подмножества
- `withscores` — возврат элементов с весами (если не указан аргумент `bylex`)

```
1  # Вывод элементов множества с сортиров
2  127.0.0.1:6379> zrange manufacturers 0
3  1) "GMC"
4  2) "Chevrolet"
5  3) "Ford"
6
7  # Вывод элементов множества и их веса
8  127.0.0.1:6379> zrange manufacturers 0
9  1) "GMC"
10 2) "10"
11 3) "Chevrolet"
12 4) "30"
13 5) "Ford"
```

```
14 6) "40"
15
16 # Вывод элементов множества до GMC (ис
17 127.0.0.1:6379> zrange manufacturers -
18 1) "Chevrolet"
19 2) "Ford"
20
21 # Вывод элементов множества после Ford
22 127.0.0.1:6379> zrange manufacturers [
23 1) "Ford"
24 2) "GMC"
25
26 # Вывод всех элементов множества с сор
27 127.0.0.1:6379> zrange manufacturers -
28 1) "Chevrolet"
29 2) "Ford"
30 3) "GMC"
```

- `zrank <key> <member> <withscore>` — Получение позиции элемента во множестве

```
1 127.0.0.1:6379> zrank manufacturers GMC
2 (integer) 0
3
4 127.0.0.1:6379> zrank manufacturers GMC
5 1) (integer) 0
6 2) "10"
```

Итого

В данной статье я вкратце рассказал о том, что такое Redis, описал наиболее используемые типы данных и основным операции с ними. В будущих статьях каждый тип данных будет рассмотрен более подробно, а так же будет описана работа с Redis в проектах на Java.

 22.10.2024  Alexander Kosarev  Redis 
Key Value, NoSQL, redis

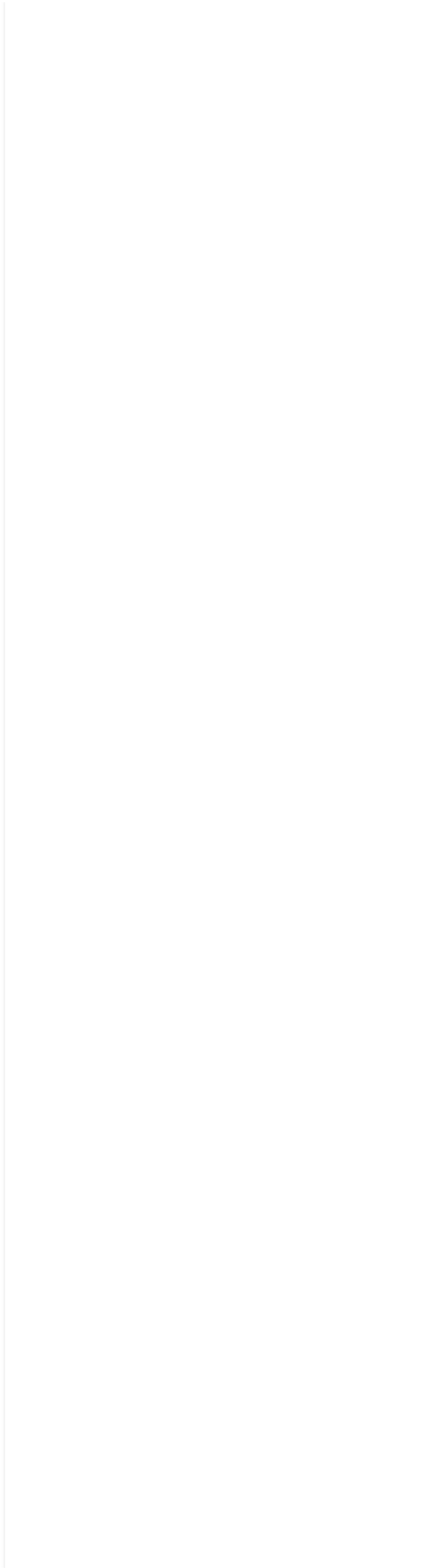
ТАКЖЕ НА MY BLOG

6 лет назад • 1 комментарий

**Тестирование
сервисов на ...**

2 года назад

**Анали
уязви
завис**



Что вы думаете? / What do you think?

10 ОТКЛИКОВ



Upvote

0
Комментарі

 Войти ▼

Начать
обсуждени

ВОЙТИ С
ПОМОЩЬЮ



Сайт работает на WordPress